

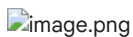
Business Case: Walmart - Confidence Interval and CLT

Table of Contents

- [Business Case: Walmart - Confidence Interval and CLT](#)
- [1. Introduction](#)
 - [? What is Walmart?](#)
 - [🎯 Objective](#)
 - [📊 About Data](#)
 - [📄 Features of the dataset:](#)
- [2. Exploratory Data Analysis](#)
 - [📄 Changing the Datatype of Columns](#)
 - [📊 Statistical Summary](#)
 - [👥 Duplicate Detection](#)
 - [✅ Sanity Check for columns](#)
 - [👤 Missing Value Analysis](#)
- [3. Univariate Analysis](#)
 - [3.1 Numerical Variables](#)
 - [3.2 Categorical Variables](#)
- [4. Bivariate Analysis](#)
 - [4.1 📊 Exploring Purchase Patterns](#)
- [5. 👤 Gender VS 🛒 Purchase Amount](#)
 - [5.1 📊 Data Visualization](#)
 - [5.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction](#)
- [6. 👤 Marital Status VS 🛒 Purchase Amount](#)
 - [6.1 📊 Data Visualization](#)
 - [6.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction](#)
- [7. 📅 Customer Age VS 🛒 Purchase Amount](#)
 - [7.1 📊 Data Visualization](#)
 - [7.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction](#)
- [8. Recommendations](#)

1. Introduction

? What is Walmart?



Walmart, founded in 1962 by Sam Walton, is a retail giant and one of the world's largest and most influential companies. Headquartered in Bentonville, Arkansas, this American multinational corporation has established itself as a global powerhouse in the retail industry. Walmart operates a vast network of hypermarkets, discount department stores, and grocery stores under various brand names across the United States and in numerous countries around the world.

Known for its "Everyday Low Prices" strategy, Walmart has redefined the retail landscape with its commitment to offering a wide range of products at affordable prices. With its extensive supply chain and efficient distribution systems, the company has played a pivotal role in shaping consumer expectations and shopping habits. Beyond retail, Walmart has also ventured into e-commerce, technology innovation, and sustainability initiatives, further solidifying its position as a key player in the modern retail ecosystem.

🎯 Objective

The objective of this project is to conduct a comprehensive analysis of customer purchase behavior, with a specific focus on purchase amounts, in relation to customer gender during the Black Friday sales event at Walmart Inc. This study aims to provide valuable insights that can assist the management team at Walmart Inc. in making data-driven decisions.

📊 About Data

The company collected the transactional data of customers who purchased products from the Walmart Stores during Black Friday. It has information of about 0.5 Million transactions during Black Friday throughout various years.

Features of the dataset:

Feature	Description
User_ID	User ID of the Customer
Product ID	Product ID of the Purchased Product
Gender	Gender of the Customer (Male/Female)
Age	Age of the Customer (in bins)
Occupation	Occupation of the Customer (Masked)
City_Category	Category of the City (A,B,C)
StayInCurrentCityYears	Number of years stay in current city
Marital_Status	Marital Status (0 - Unmarried / 1 - Married)
ProductCategory	Product Category (Masked)
Purchase	Purchase Amount

Note:-We make all plots & graph with same color palette as Walmart uses for thier logo.

2. Exploratory Data Analysis

```
1 #importing libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from scipy.stats import t
7 import warnings
8 warnings.filterwarnings('ignore')
9 import copy
```

```
1 # loading the dataset
2 from google.colab import files
3 uploaded = files.upload()
4
```

[Choose Files](#) walmart_dataset.csv

walmart_dataset.csv(text/csv) - 23027994 bytes, last modified: 5/13/2026 - 100% done
Saving walmart_dataset.csv to walmart_dataset.csv

```
1 df=pd.read_csv('walmart_dataset.csv')
```

```
1 df.head()
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422

```
1 df.tail()
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
550063	1006033	P00372445	M	51-55	13	B	1	1	20	368
550064	1006035	P00375436	F	26-35	1	C	3	0	20	371
550065	1006036	P00375436	F	26-35	15	B	4+	1	20	137

```
1 df.shape
```

```
(550068, 10)
```

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID                550068 non-null  int64
1   Product_ID             550068 non-null  object
```

```
2 Gender 550068 non-null object
3 Age 550068 non-null object
4 Occupation 550068 non-null int64
5 City_Category 550068 non-null object
6 Stay_In_Current_City_Years 550068 non-null object
7 Marital_Status 550068 non-null int64
8 Product_Category 550068 non-null int64
9 Purchase 550068 non-null int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
```

```
1 df.describe()
```

	User_ID	Occupation	Marital_Status	Product_Category	Purchase
count	5.500680e+05	550068.000000	550068.000000	550068.000000	550068.000000
mean	1.003029e+06	8.076707	0.409653	5.404270	9263.968713
std	1.727592e+03	6.522660	0.491770	3.936211	5023.065394
min	1.000001e+06	0.000000	0.000000	1.000000	12.000000
25%	1.001516e+06	2.000000	0.000000	1.000000	5823.000000
50%	1.003077e+06	7.000000	0.000000	5.000000	8047.000000
75%	1.004478e+06	14.000000	1.000000	8.000000	12054.000000
max	1.006040e+06	20.000000	1.000000	20.000000	23961.000000

Insights

- User_ID** - Among **5,50,068** transactions there are **5891** unique user_id, indicating same customers buying multiple products.
- Product_ID** - Among **5,50,068** transactions there are **3631** unique products,with the product having the code **P00265242** being the **highest seller**, with a maximum of **1,880 units** sold.
- Gender** - Out of **5,50,068** transactions, 4,14,259 (**nearly 75%**) were done by male gender indicating a significant disparity in purchase behavior between males and females during the Black Friday event.
- Age** - We have **7** unique age groups in the dataset. **26 - 35** Age group has maximum of **2,19,587** transactions. We will analyse this feature in detail in future
- Stay_In_Current_City_Years** - Customers with **1** year of stay in current city accounted to maximum of **1,93,821** transactions among all the other customers with (0,2,3,4+) years of stay in current city
- Marital_Status** - **59%** of the total transactions were done by **Unmarried Customers** and **41%** by **Married Customers**.

👥 Duplicate Detection

```
1 df.duplicated().value_counts()
```

	count
False	550068

dtype: int64

Insights

- There are no duplicate entries in the dataset

✅ Sanity Check for columns

```
1 # checking the unique values for columns
2 for i in df.columns:
3     print('Unique Values in',i,'column are :-')
4     print(df[i].unique())
5     print('-'*70)
```

```
Unique Values in User_ID column are :-
[1000001 1000002 1000003 ... 1004113 1005391 1001529]
-----
Unique Values in Product_ID column are :-
['P00069042' 'P00248942' 'P00087842' ... 'P00370293' 'P00371644'
 'P00370853']
-----
Unique Values in Gender column are :-
['F' 'M']
-----
Unique Values in Age column are :-
['0-17' '55+' '26-35' '46-50' '51-55' '36-45' '18-25']
```

Unique Values in Occupation column are :-
[10 16 15 7 20 9 1 12 17 0 3 4 11 8 19 2 18 5 14 13 6]

Unique Values in City_Category column are :-
['A' 'C' 'B']

Unique Values in Stay_In_Current_City_Years column are :-
['2' '4+' '3' '1' '0']

Unique Values in Marital_Status column are :-
[0 1]

Unique Values in Product_Category column are :-
[3 1 12 8 5 4 2 6 14 11 13 15 7 16 18 10 17 9 20 19]

Unique Values in Purchase column are :-
[8370 15200 1422 ... 135 123 613]

Insights

- The dataset does not contain any abnormal values.
- We will convert the 0,1 in Marital Status column as married and unmarried

```
1 #replacing the values in marital_status column
2
3 df['Marital_Status'] = df['Marital_Status'].replace({0:'Unmarried',1:'Married'})
4 df['Marital_Status'].unique()
```

```
array(['Unmarried', 'Married'], dtype=object)
```

Missing Value Analysis

```
1 df.isnull().sum()
```

	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

dtype: int64

Insights

- The dataset does not contain any missing values.

2. Detect Null values and outliers

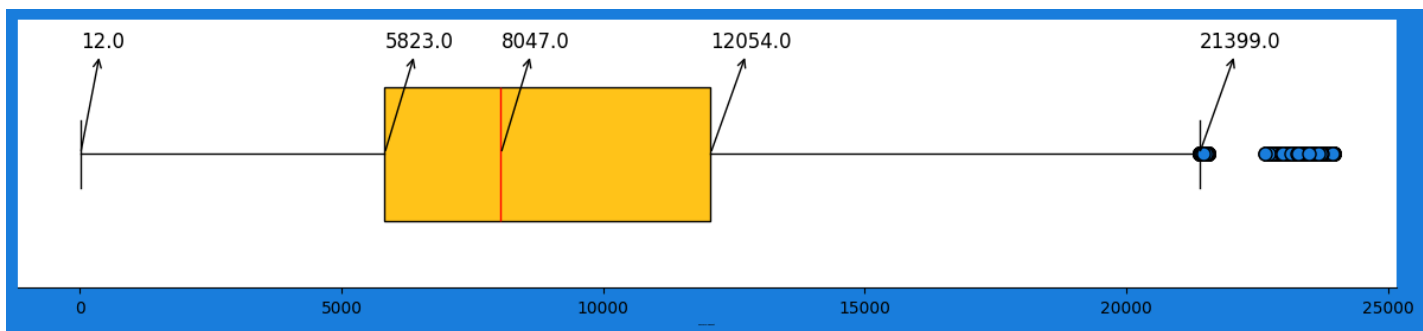
a. Find the outliers for every continuous variable in the dataset

```
1 from matplotlib.colors import colorConverter
2 #setting the plot style
3
4 fig = plt.figure(figsize = (15,10),facecolor='#1b81dd')
5                                     #creating box plot for purchase amount
6
7 ax1 = fig.add_subplot(gs[1,0])
8 boxplot = ax1.boxplot(x = df['Purchase'],vert = False,patch_artist = True,widths = 0.5)
9
10 # Customize box and whisker colors
11 boxplot['boxes'][0].set(facecolor='#FFC31A')
12
13 # Customize median line
14 boxplot['medians'][0].set(color='red')
```

```

15
16 # Customize outlier markers
17 for flier in boxplot['fliers']:
18     flier.set(marker='o', markersize=8, markerfacecolor= "#1b81dd")
19
20 #removing the axis lines
21 for s in ['top','left','right']:
22     ax1.spines[s].set_visible(False)
23
24 #adding 5 point summary annotations
25 info = [i.get_xdata() for i in boxplot['whiskers']] #getting the upperlimit,Q1,Q3 and lowerlimit
26
27 median = df['Purchase'].quantile(0.5) #getting Q2
28
29 for i,j in info: #using i,j here because of the output type of info list comprehension
30
31     ax1.annotate(text = f"{i:.1f}", xy = (i,1), xytext = (i,1.4),fontsize = 12,
32                 arrowprops= dict(arrowstyle="<-", lw=1, connectionstyle="arc,rad=0"))
33
34     ax1.annotate(text = f"{j:.1f}", xy = (j,1), xytext = (j,1.4),fontsize = 12,
35                 arrowprops= dict(arrowstyle="<-", lw=1, connectionstyle="arc,rad=0"))
36
37 #adding the median separately because it was included in info list
38 ax1.annotate(text = f"{median:.1f}",xy = (median,1),xytext = (median + 1,1.4),fontsize = 12,
39             arrowprops= dict(arrowstyle="<-", lw=1, connectionstyle="arc,rad=0"))
40
41 #removing y-axis ticks
42 ax1.set_yticks([])
43
44 #adding axis label
45 ax1.set_xlabel('Purchase Amount',fontweight = 'bold',fontsize = 1)
46

```



Calculating the Number of Outliers

- As seen above, Purchase amount over 21399 is considered as outlier. We will count the number of outliers as below

```
1 len(df.loc[df['Purchase'] > 21399, 'Purchase'])
```

2677

Insights

Outliers

- There are total of 2677 outliers which is roughly 0.48% of the total data present in purchase amount. We will not remove them as it indicates a broad range of spending behaviors during the sale, highlighting the importance of tailoring marketing strategies to both regular and high-value customers to maximize revenue.

Distribution

- Data suggests that the majority of customers spent between 5,823 USD and 12,054 USD, with the median purchase amount being 8,047 USD.
- The lower limit of 12 USD while the upper limit of 21,399 USD reveal significant variability in customer spending

b. Remove/clip the data between the 5 percentile and 95 percentile

```

1 # 1. Calculate the 5th and 95th percentile values for the 'Purchase' column
2 lower_limit = np.percentile(df['Purchase'], 5)

```

```

3 upper_limit = np.percentile(df['Purchase'], 95)
4
5 # 2. Clip the 'Purchase' data
6 # Values below lower_limit become lower_limit
7 # Values above upper_limit become upper_limit
8 df['Purchase_Clippped'] = np.clip(df['Purchase'], lower_limit, upper_limit)
9
10 # we can verify the new column or overwrite the original 'Purchase' column if desired
11 df['Purchase'] = np.clip(df['Purchase'], lower_limit, upper_limit)

```

```

1 # 'Purchase' has already been clipped in the previous cell, this demonstrates the loop structure.
2 continuous_variables_to_clip = ['Purchase']
3
4 # Apply clipping to each continuous variable in the list
5 for col_name in continuous_variables_to_clip:
6     # Find the 5th and 95th percentile values for the current column
7     lower_limit = df[col_name].quantile(0.05)
8     upper_limit = df[col_name].quantile(0.95)
9
10    # Clip the data in the current column
11    df[col_name] = np.clip(df[col_name], lower_limit, upper_limit)
12
13 print("Data clipped between 5th and 95th percentiles for selected columns.")

```

Data clipped between 5th and 95th percentiles for selected columns.

```
1 df.head()
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase	Pur
0	1000001	P00069042	F	0-17	10	A	2	Unmarried	3	8370	
1	1000001	P00248942	F	0-17	10	A	2	Unmarried	1	15200	
2	1000001	P00087842	F	0-17	10	A	2	Unmarried	12	1984	

```
1 df
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969
...	...	...	...	...	...	...	...	...	...	...
550063	1006033	P00372445	M	51-55	13	B	1	1	20	368
550064	1006035	P00375436	F	26-35	1	C	3	0	20	371

3. Data Exploration

a. What products are different age groups buying?

```

1 #setting the plot style
2 fig = plt.figure(figsize = (15,7),facecolor='#e5e5ff')
3 gs = fig.add_gridspec(1,2,width_ratios=[0.6, 0.4])
4
5     # creating bar chart for age distribution
6
7 ax0 = fig.add_subplot(gs[0,0])
8 temp = df['Age'].value_counts()
9 color_map = ["#0052E3", "#2A65C2", '#5578A0', '#808B7F', '#AA9D5D', '#D5B03C', '#FFC31A']
10 ax0.bar(x=temp.index,height = temp.values,color = color_map,zorder = 2)
11
12 #adding the value_counts
13 for i in temp.index:
14     ax0.text(i,temp[i]+5000,temp[i],{'font':'serif','size' : 10},ha = 'center',va = 'center')

```

```

15
16 #adding grid lines
17 ax0.grid(color = 'black',linestyle = '--', axis = 'y', zorder = 0, dashes = (5,10))
18
19 #removing the axis lines
20 for s in ['top','left','right']:
21     ax0.spines[s].set_visible(False)
22
23 #adding axis label
24 ax0.set_ylabel('Count',fontweight = 'bold',fontsize = 12)
25 ax0.set_xlabel('Age Group',fontweight = 'bold',fontsize = 12,color='#FFC31A')
26 ax0.set_xticklabels(temp.index,fontweight = 'bold')
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

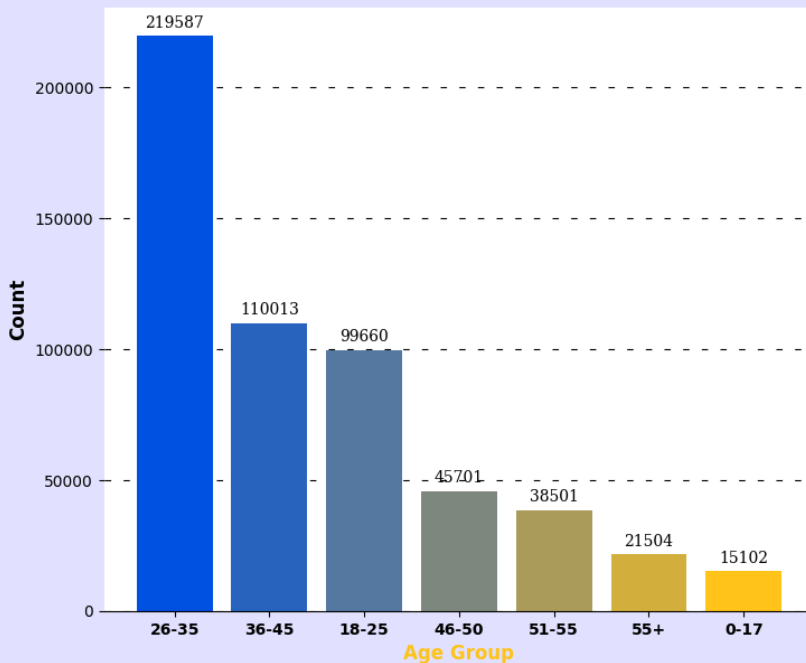
```

```

#creating a info table for age
ax1 = fig.add_subplot(gs[0,1])
age_info = age_info = [['26-35', '40%'], ['36-45', '20%'], ['18-25', '18%'], ['46-50', '8%'], ['51-55', '7%'], ['55+', '4%'],
                        ['0-17', '3%']]
color_2d = [['#0052E3', '#FFFFFF'], ['#2A65C2', '#FFFFFF'], ['#5578A0', '#FFFFFF'], ['#808B7F', '#FFFFFF'], ['#AA9D5D', '#FFFFFF'],
            ['#D5B03C', '#FFFFFF'], ['#FFC31A', '#FFFFFF']]
table = ax1.table(cellText = age_info, cellColours=color_2d, cellLoc='center', colLabels = ['Age Group', 'Percent Dist.'],
                  colLoc = 'center', bbox = [0, 0, 1, 1])
table.set_fontsize(15)
#removing axis
ax1.axis('off')
#setting title for visual
fig.suptitle('Customer Age Distribution',font = 'serif', size = 18, weight = 'bold',color='#0052E3')
plt.show()

```

Customer Age Distribution



Age Group	Percent Dist.
26-35	40%
36-45	20%
18-25	18%
46-50	8%
51-55	7%
55+	4%
0-17	3%

Insights

- The age group of **26-35** represents the largest share of Walmart's Black Friday sales, accounting for **40%** of the sales. This suggests that the young and middle-aged adults are the **most active and interested** in shopping for **deals and discounts**.

- The 36-45 and 18-25 age groups are the second and third largest segments, respectively, with 20% and 18% of the sales. This indicates that Walmart has a diverse customer base that covers different life stages and preferences..
- The 46-50, 51-55, 55+, and 0-17 age groups are the smallest customer segments, with less than 10% of the total sales each. This implies that Walmart may need to improve its marketing strategies and product offerings to attract more customers from these age groups, especially the seniors and the children.

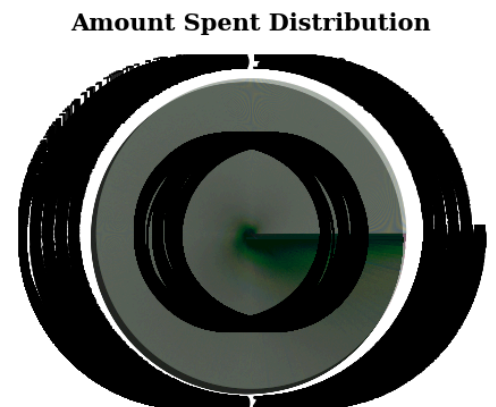
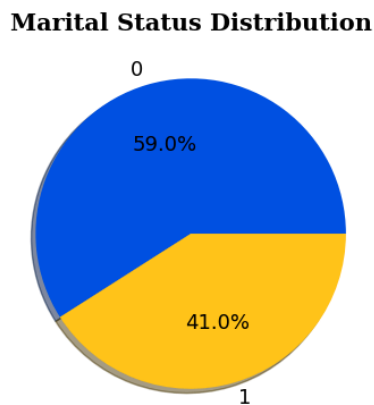
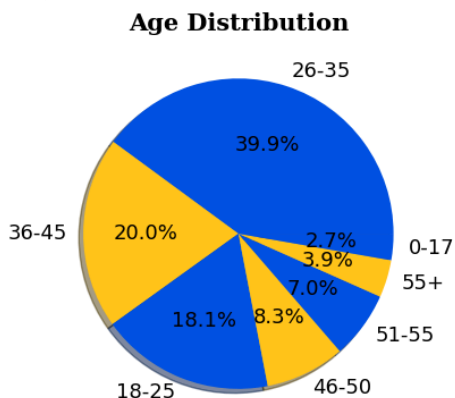
b. Is there a relationship between age, marital status, and the amount spent?

```

1 #setting the plot style
2 fig = plt.figure(figsize = (15,12))
3 gs = fig.add_gridspec(1,3)
4
5                                     # creating pie chart for age distribution
6 ax0 = fig.add_subplot(gs[0,0])
7
8 color_map = ["#0052E3", "#FFC31A"]
9 ax0.pie(df['Age'].value_counts().values,labels = df['Age'].value_counts().index,autopct = '%.1f%%',
10        shadow = True,colors = color_map,textprops={'fontsize': 13, 'color': 'black'})
11
12 #setting title for visual
13 ax0.set_title('Age Distribution',{ 'font':'serif', 'size':15,'weight':'bold'})
14
15                                     # creating pie chart for marital status
16 ax1 = fig.add_subplot(gs[0,1])
17
18 color_map = ["#0052E3", "#FFC31A"]
19 ax1.pie(df['Marital_Status'].value_counts().values,labels = df['Marital_Status'].value_counts().index,autopct = '%.1f%%',
20        shadow = True,colors = color_map,textprops={'fontsize': 13, 'color': 'black'})
21
22 #setting title for visual
23 ax1.set_title('Marital Status Distribution',{ 'font':'serif', 'size':15,'weight':'bold'})
24
25                                     # creating pie chart for amount spent
26 ax2 = fig.add_subplot(gs[0,2])
27
28 color_map = ["#0052E3", "#FFC31A"]
29 ax2.pie(df['Purchase'].value_counts().values,labels = df['Purchase'].value_counts().index,autopct = '%.1f%%',
30        shadow = True,colors = color_map,textprops={'fontsize': 13, 'color': 'black'})
31
32 #setting title for visual
33 ax2.set_title('Amount Spent Distribution',{ 'font':'serif', 'size':15,'weight':'bold'})
34

```

Text(0.5, 1.0, 'Amount Spent Distribution')

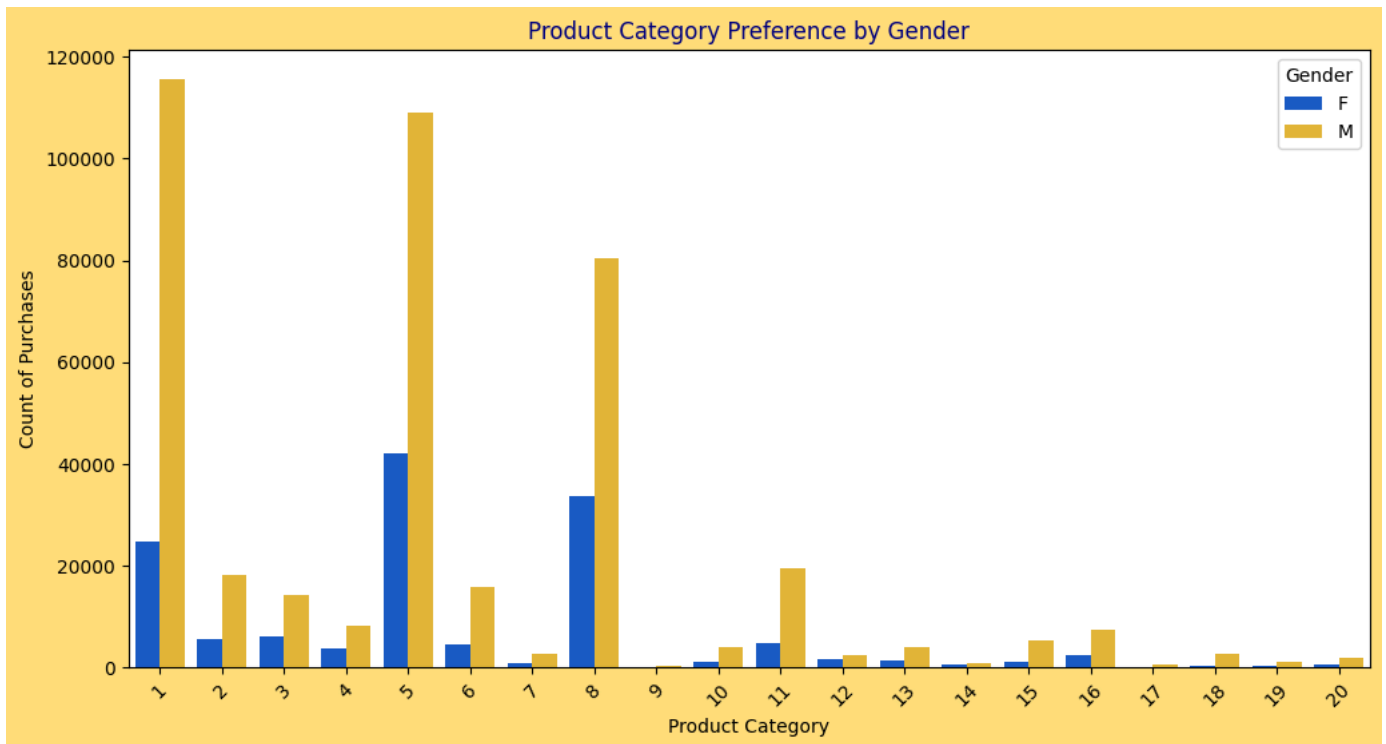


Insights

1. Age Distribution - Data indicates a significant disparity in Age behavior between 26-35 Age group and others.
2. Marital Status - Given that unmarried customers account for a higher percentage of transactions, it may be worthwhile to consider specific marketing campaigns or promotions that appeal to this group.
3. Amount Spent- Product category 1,5 & 8 has highest number of Distribution as compared to product category 9,14 & 17 which is lowest.

c. Are there preferred product categories for different genders?

```
1 from PIL.Image import Palette
2 # Set the figure size for better readability
3 plt.figure(figsize=(12, 6),facecolor='#ffdf7b')
4
5 # Create a count plot to compare product categories by gender
6 sns.countplot(data=df, x='Product_Category', hue='Gender',palette= ["#0052E3", "#FFC31A"])
7
8 # Add titles and labels
9 plt.title('Product Category Preference by Gender',color='#00007f')
10 plt.xlabel('Product Category')
11 plt.ylabel('Count of Purchases')
12 plt.xticks(rotation=45) # Rotate labels if category names are long
13
14 plt.show()
15
```



🔍 Insights

- Dominant Categories: Categories 1, 5, and 8 drive the highest volume of total purchases.
- Gender Gap: Males (M, orange) vastly outnumber females (F, blue) in purchase counts across almost every single category.
- Peak Category: Category 1 registers the highest individual purchase count, driven heavily by male buyers.
- Widest Disparity: Category 5 shows the largest absolute gap in volume between male and female purchasers.
- Low-Volume Segments: Categories 9 through 20 show minimal consumer engagement, maintaining low purchase counts for both genders.
- Female Preferences: Females show their highest relative engagement in categories 1, 5, and 8, mirroring the male trend but at a lower scale.

4. How does gender affect the amount spent?

📊 Data Visualizations

```
1 #creating a df for purchase amount vs gender
2 temp = df.groupby('Gender')['Purchase'].agg(['sum','count']).reset_index()
3
4 #calculating the amount in billions
5 temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)
6
7 #calculating percentage distribution of purchase amount
8 temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)
9
10 #calculating per purchase amount
11 temp['per_purchase'] = round(temp['sum']/temp['count'])
12
13 #renaming the gender
```

```

14 temp['Gender'] = temp['Gender'].replace({'F':'Female','M':'Male'})
15
16 temp

```

	Gender	sum	count	sum_in_billions	%sum	per_purchase	
0	Female	1186232642	135809	1.19	0.233	8735.0	
1	Male	3909580100	414259	3.91	0.767	9438.0	



Next steps:

[Generate code with temp](#)

[New interactive sheet](#)

```

1 #setting the plot style
2 fig = plt.figure(figsize = (15,14))
3 gs = fig.add_gridspec(3,2,height_ratios =[0.10,0.4,0.5])
4
5
6                                     #Distribution of Purchase Amount
7 ax = fig.add_subplot(gs[0,:])
8
9 #plotting the visual
10 ax.barh(temp.loc[0,'Gender'],width = temp.loc[0,'%sum'],color = "#0052E3",label = 'Female')
11 ax.barh(temp.loc[1,'Gender'],width = temp.loc[1,'%sum'],left =temp.loc[0,'%sum'], color = "#FFC31A",label = 'Male' )
12
13 #inserting the text
14 txt = [0.0] #for left parameter in ax.text()
15
16 for i in temp.index:
17     #for amount
18     ax.text(temp.loc[i,'%sum']/2 + txt[0],0.15,f"${temp.loc[i,'sum_in_billions']} Billion",
19            va = 'center', ha='center',fontsize=18, color='white')
20
21     #for gender
22     ax.text(temp.loc[i,'%sum']/2 + txt[0],- 0.20 ,f"{temp.loc[i,'Gender']}",
23            va = 'center', ha='center',fontsize=14, color='white')
24
25     txt += temp.loc[i,'%sum']
26
27 #removing the axis lines
28 for s in ['top','left','right','bottom']:
29     ax.spines[s].set_visible(False)
30
31 #customizing ticks
32 ax.set_xticks([])
33 ax.set_yticks([])
34 ax.set_xlim(0,1)
35
36 #plot title
37 ax.set_title('Gender-Based Purchase Amount Distribution',{ 'font':'serif', 'size':15,'weight':'bold'})
38
39
40                                     #Distribution of Purchase Amount per Transaction
41
42 ax1 = fig.add_subplot(gs[1,0])
43
44 color_map = ["#0052E3", "#FFC31A"]
45
46 #plotting the visual
47 ax1.bar(temp['Gender'],temp['per_purchase'],color = color_map,zorder = 2,width = 0.3)
48
49 #adding average transaction line
50 avg = round(df['Purchase'].mean())
51
52 ax1.axhline(y = avg, color = 'red', zorder = 0,linestyle = '--')
53
54 #adding text for the line
55 ax1.text(0.4,avg + 300, f"Avg. Transaction Amount ${avg:.0f}",
56         { 'font':'serif','size' : 12},ha = 'center',va = 'center')
57
58 #adjusting the ylimits
59 ax1.set_ylim(0,11000)
60
61 #adding the value_counts
62 for i in temp.index:
63     ax1.text(temp.loc[i,'Gender'],temp.loc[i,'per_purchase']/2,f"${temp.loc[i,'per_purchase']:.0f}",
64            { 'font':'serif','size' : 12,'color':'white','weight':'bold' },ha = 'center',va = 'center')
65
66 #adding grid lines
67 ax1.grid(color = 'black',linestyle = '--', axis = 'y', zorder = 0, dashes = (5,10))
68
69 #removing the axis lines
70 for s in ['top','left','right']:
71     ax1.spines[s].set_visible(False)
72
73 #adding axis label
74 ax1.set_ylabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
75 ax1.set_xticklabels(temp['Gender'],fontweight = 'bold',fontsize = 12)

```

```

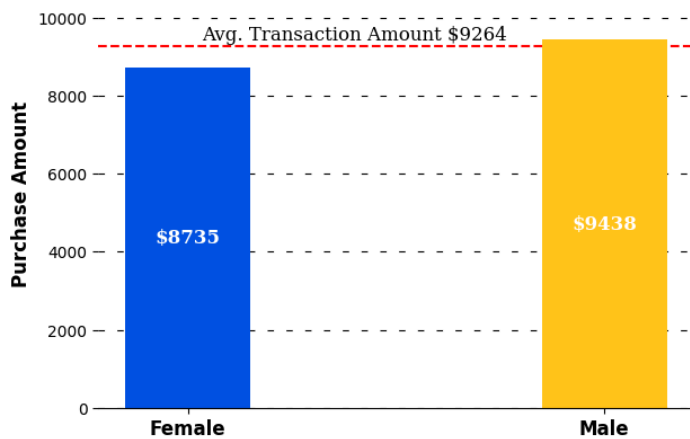
76
77 #setting title for visual
78 ax1.set_title('Average Purchase Amount per Transaction',{'font':'serif', 'size':15,'weight':'bold'})
79
80                                     # creating pie chart for gender disribution
81 ax2 = fig.add_subplot(gs[1,1])
82
83 color_map = ["#0052E3", "#FFC31A"]
84 ax2.pie(temp['count'],labels = temp['Gender'],autopct = '%.1f%%',
85         shadow = True,colors = color_map,wedgeprops = {'linewidth': 5},textprops={'fontsize': 13, 'color': 'black'})
86
87 #setting title for visual
88 ax2.set_title('Gender-Based Transaction Distribution',{'font':'serif', 'size':15,'weight':'bold'})
89
90                                     # creating kdeplot for purchase amount distribution
91
92 ax3 = fig.add_subplot(gs[2,:])
93
94 #plotting the kdeplot
95 sns.kdeplot(data = df, x = 'Purchase', hue = 'Gender', palette = color_map,fill = True, alpha = 1,ax = ax3)
96
97 #removing the axis lines
98 for s in ['top','left','right']:
99     ax3.spines[s].set_visible(False)
100
101 # adjusting axis labels
102 ax3.set_yticks([])
103 ax3.set_ylabel('')
104 ax3.set_xlabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
105
106 #setting title for visual
107 ax3.set_title('Purchase Amount Distribution by Gender',{'font':'serif', 'size':15,'weight':'bold'})
108
109 plt.show()

```

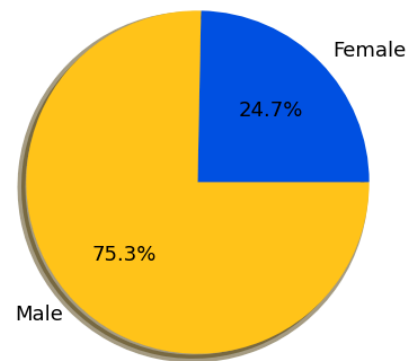
Gender-Based Purchase Amount Distribution



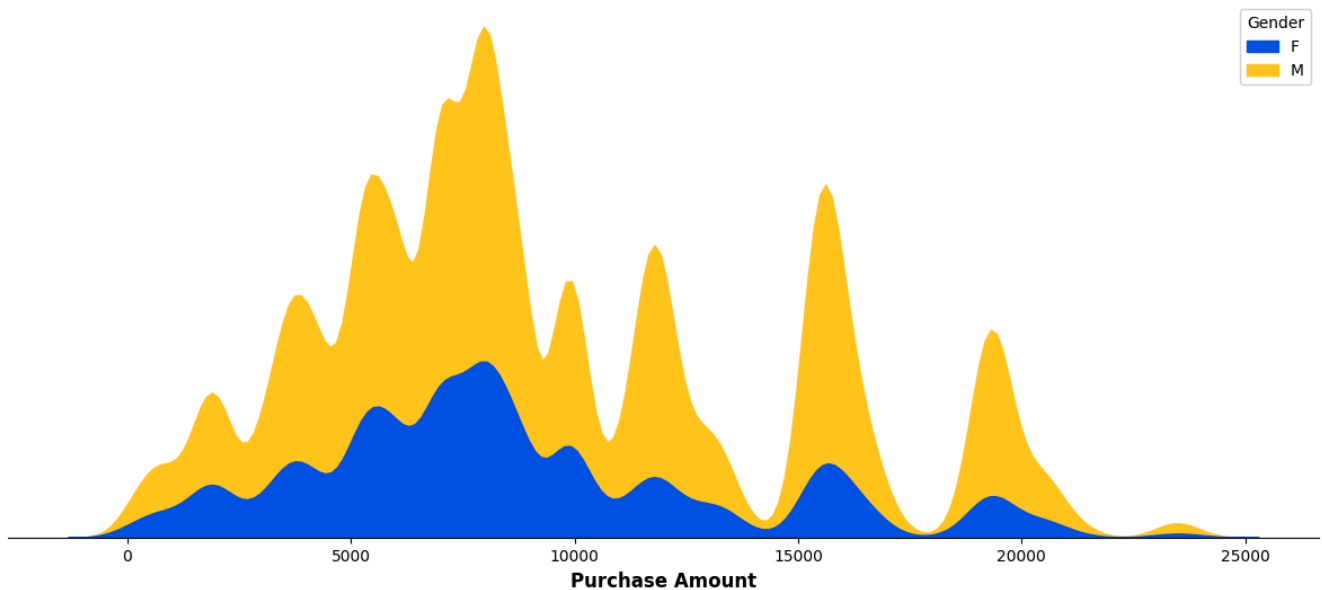
Average Purchase Amount per Transaction



Gender-Based Transaction Distribution



Purchase Amount Distribution by Gender



Insights

1. Total Sales and Transactions Comparison

- The total purchase amount and number of transactions by male customers was **more** than **three times** the amount and transactions by female customers indicating that they had a more significant impact on the Black Friday sales.

2. Average Transaction Value

- The average purchase amount per transaction was slightly higher for male customers than female customers (**\$9,438 vs \$8,735**).

3. Distribution of Purchase Amount

- As seen above, the purchase amount for both the genders is **not normally distributed**.

Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use **Central Limit Theorem**. It states the distribution of sample means will approximate a normal distribution, regardless of the underlying population distribution

2. Step 2 - Building Confidence Interval

- After building CLT curve, we will create a confidence interval predicting population mean at 99%,95% and 90% Confidence level.

Note - We will use different sample sizes of [300,3000,30000]

```
1 #creating a function to calculate confidence interval
2
3 def confidence_interval(data,ci):
4     #converting the list to series
5     l_ci = (100-ci)/2
6     u_ci = (100+ci)/2
7
8     #calculating lower limit and upper limit of confidence interval
9     interval = np.percentile(data,[l_ci,u_ci]).round(0)
10
11     return interval
```

```
1 # defining a function for plotting the visual for given confidence interval
2
3 def plot(ci):
4
5     #setting the plot style
6     fig = plt.figure(figsize = (15,8))
7     gs = fig.add_gridspec(2,2)
8
9     #creating separate data frames for each gender
10    df_male = df.loc[df['Gender'] == 'M','Purchase']
11    df_female = df.loc[df['Gender'] == 'F','Purchase']
12
13    #sample sizes and corresponding plot positions
14    sample_sizes = [(300,0,0),(3000,0,1),(30000,1,0),(50000,1,1)]
15
16    #number of samples to be taken from purchase amount
17    bootstrap_samples = 20000
18
19    male_samples = {}
20    female_samples = {}
21
22    for i,x,y in sample_sizes:
23        male_means = [] #list for collecting the means of male sample
24        female_means = [] #list for collecting the means of female sample
25
26        for j in range(bootstrap_samples):
27
28            #creating random 3000 samples of i sample size
29            male_bootstrapped_samples = np.random.choice(df_male,size = i)
30            female_bootstrapped_samples = np.random.choice(df_female,size = i)
31
32            #calculating mean of those samples
33            male_sample_mean = np.mean(male_bootstrapped_samples)
34            female_sample_mean = np.mean(female_bootstrapped_samples)
35
36            #appending the mean to the list
37            male_means.append(male_sample_mean)
38            female_means.append(female_sample_mean)
39
40        #storing the above sample generated
41        male_samples[f'{ci}%_{i}'] = male_means
42        female_samples[f'{ci}%_{i}'] = female_means
43
44        #creating a temporary dataframe for creating kdeplot
45        temp_df = pd.DataFrame(data = {'male_means':male_means,'female_means':female_means})
46
47        #plotting kdeplots
48
49        #plot position
50        ax = fig.add_subplot(gs[x,y])
51
52        #plots for male and female
53        sns.kdeplot(data = temp_df,x = 'male_means',color = "#0052E3",fill = True, alpha = 0.5,ax = ax,label = 'Male')
54        sns.kdeplot(data = temp_df,x = 'female_means',color = "#FFC31A",fill = True, alpha = 0.5,ax = ax,label = 'Female')
55
56        #calculating confidence intervals for given confidence level(ci)
57        m_range = confidence_interval(male_means,ci)
58        f_range = confidence_interval(female_means,ci)
59
60        #plotting confidence interval on the distribution
61        for k in m_range:
62            ax.axvline(x = k,ymax = 0.9, color = "#0052E3",linestyle = '--')
63
64        for k in f_range:
65            ax.axvline(x = k,ymax = 0.9, color = "#FFC31A",linestyle = '--')
66
67        #removing the axis lines
68        for s in ['top','left','right']:
69            ax.spines[s].set_visible(False)
```

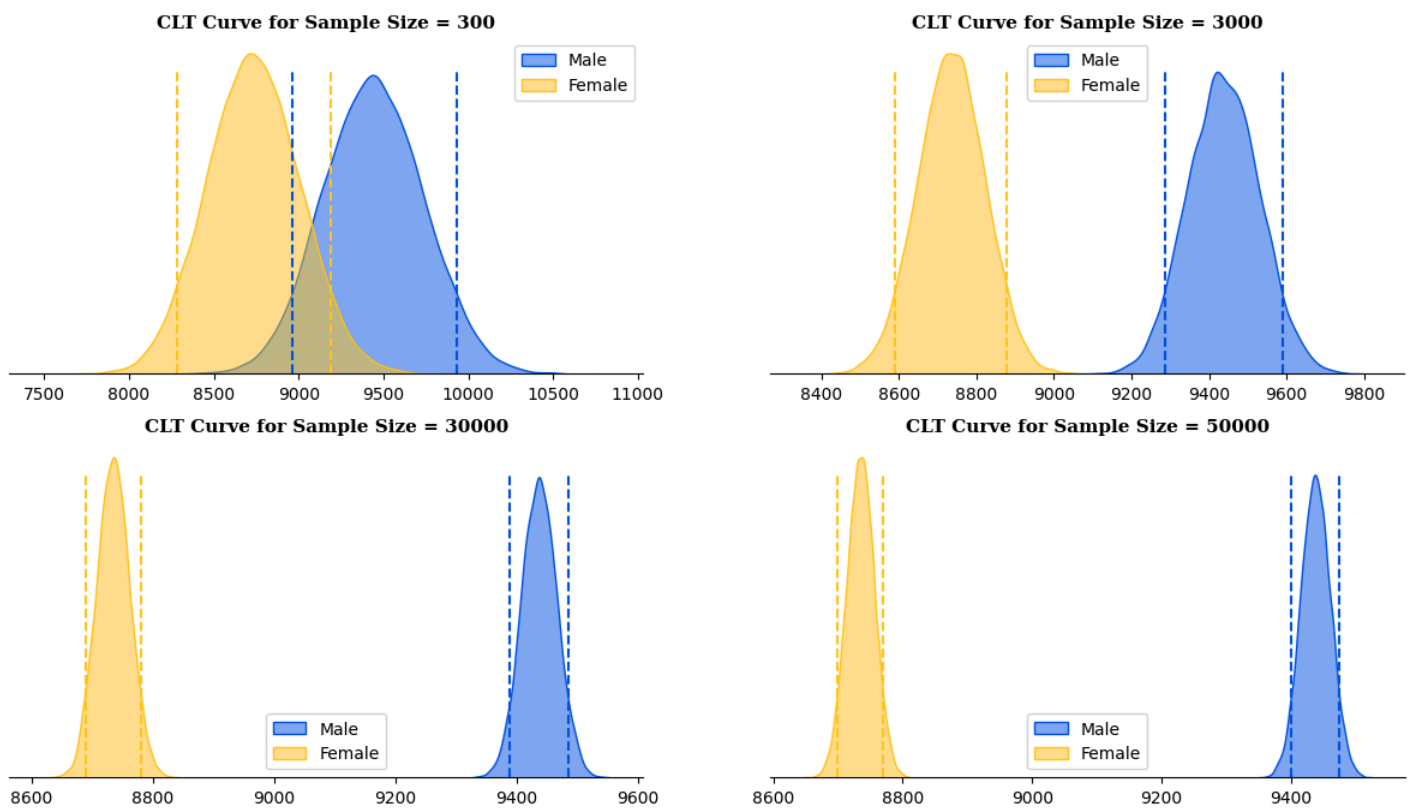
```

70
71     # adjusting axis labels
72     ax.set_yticks([])
73     ax.set_ylabel('')
74     ax.set_xlabel('')
75
76     #setting title for visual
77     ax.set_title(f'CLT Curve for Sample Size = {i}', {'font':'serif', 'size':11, 'weight':'bold'})
78
79     plt.legend()
80
81 #setting title for visual
82 fig.suptitle(f'{ci}% Confidence Interval', font = 'serif', size = 18, weight = 'bold')
83
84 plt.show()
85
86 return male_samples, female_samples

```

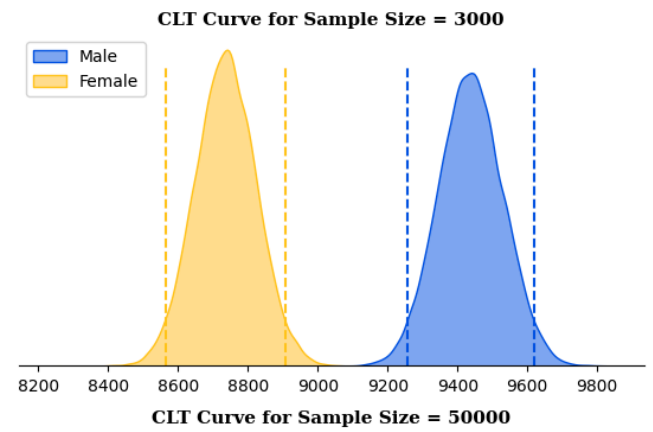
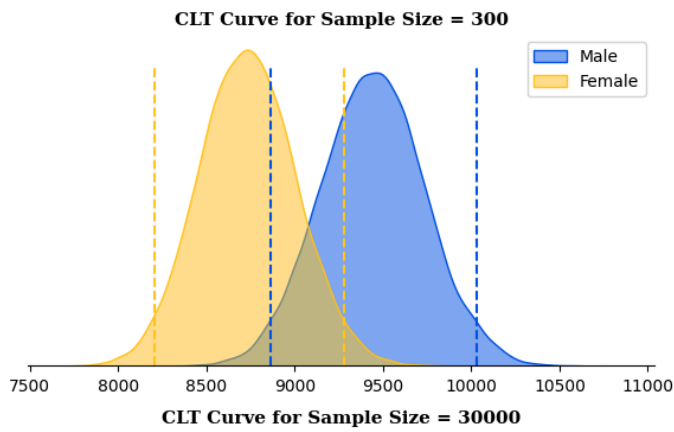
```
1 m_samp_90, f_samp_90 = plot(90)
```

90% Confidence Interval



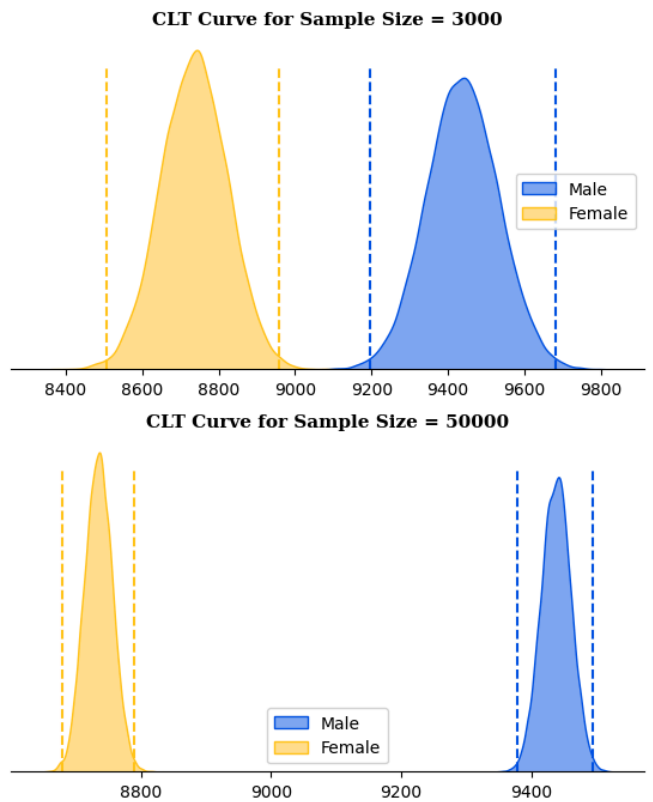
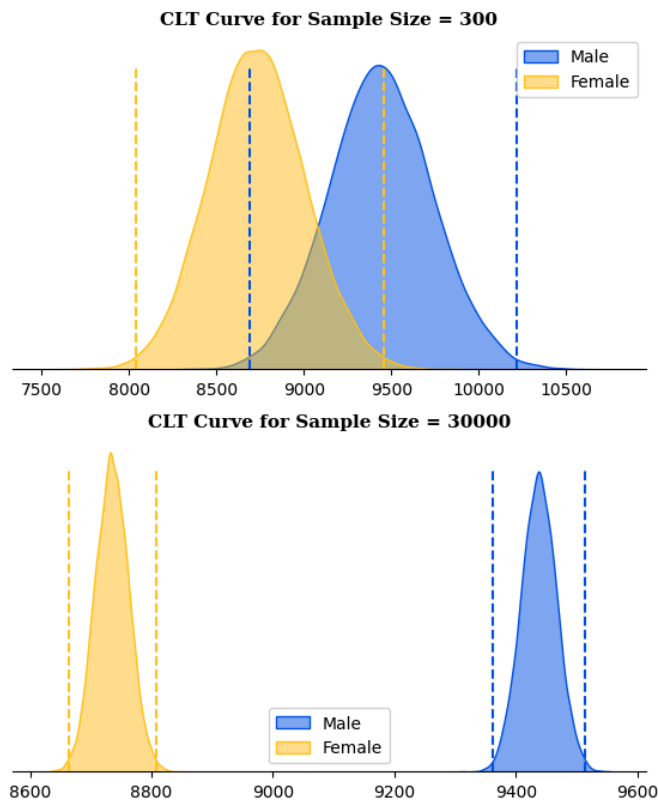
```
1 m_samp_95, f_samp_95 = plot(95)
```

95% Confidence Interval



1 m_samp_99,f_samp_99 = plot(99)

99% Confidence Interval



✓ a. From the above calculated CLT answer the following questions.

i. Is the confidence interval computed using the entire dataset wider for one of the genders? Why is this the case?

Looking at the distribution of sample means for the largest sample size (50,000), the confidence interval for the average purchase amount for females tends to be wider than for males.

This is primarily due to the higher variability (standard deviation) in female purchase amounts compared to males in the original dataset (female standard deviation approx. 5175, male standard deviation approx. 4976). Even when drawing samples of the same size, a higher underlying population standard deviation leads to a wider distribution of sample means, and consequently, a wider confidence interval for the population mean.

ii. How is the width of the confidence interval affected by the sample size?

As the sample size increases, the width of the confidence interval decreases. This can be clearly observed across the plots: the distributions of sample means become progressively narrower and more concentrated around their respective true means as the sample size grows from 300 to 50,000. A larger sample size provides more information about the population, leading to a more precise estimate of the population mean and thus a tighter confidence interval.

iii. Do the confidence intervals for different sample sizes overlap?

```
1 fig = plt.figure(figsize = (20,10))
2 gs = fig.add_gridspec(3,1)
3
4 for i,j,k,l in [(m_samp_90,f_samp_90,90,0),(m_samp_95,f_samp_95,95,1),(m_samp_99,f_samp_99,99,2)]:
5     #list for collecting ci for given cl
6     m_ci = ['Male']
7     f_ci = ['Female']
8     #finding ci for each sample size (males)
9     for m in i:
10         m_range = confidence_interval(i[m],k)
11         m_ci.append(f"CI = ${m_range[0]:.0f} - ${m_range[1]:.0f}, Range = {(m_range[1] - m_range[0]):.0f}")
12
13     #finding ci for each sample size (females)
14     for f in j:
15         f_range = confidence_interval(j[f],k)
16         f_ci.append(f"CI = ${f_range[0]:.0f} - ${f_range[1]:.0f}, Range = {(f_range[1] - f_range[0]):.0f}")
17
18     #plotting the summary
19     ax = fig.add_subplot(gs[l])
20
21     #contents of the table
22     ci_info = [m_ci,f_ci]
23
24     #plotting the table
25     table = ax.table(cellText = ci_info, cellLoc='center',
26                     colLabels = ['Gender','Sample Size = 300','Sample Size = 3000','Sample Size = 30000','Sample Size = 50000'],
27                     colLoc = 'center',colWidths = [0.05,0.2375,0.2375,0.2375,0.2375],bbox = [0, 0, 1, 1])
28
29     table.set_fontsize(13)
30
31     #removing axis
32     ax.axis('off')
33
34     #setting title
35     ax.set_title(f"{k}% Confidence Interval Summary",{ 'font':'serif', 'size':14,'weight':'bold'})
```


90% Confidence Interval Summary

Gender	Sample Size = 300	Sample Size = 3000	Sample Size = 30000	Sample Size = 50000
Male	CI = 8958 – 9924, Range = 966	CI = 9286 – 9590, Range = 304	CI = 9389 – 9486, Range = 97	CI = 9400 – 9475, Range = 75
Female	CI = 8292 – 9192, Range = 900	CI = 8590 – 8877, Range = 287	CI = 8689 – 8780, Range = 91	CI = 8700 – 8770, Range = 70

95% Confidence Interval Summary

Gender	Sample Size = 300	Sample Size = 3000	Sample Size = 30000	Sample Size = 50000
Male	CI = 8860 – 10019, Range = 1159	CI = 9257 – 9619, Range = 362	CI = 9380 – 9494, Range = 114	CI = 9392 – 9482, Range = 90
Female	CI = 8204 – 9288, Range = 1084	CI = 8564 – 8906, Range = 342	CI = 8680 – 8788, Range = 108	CI = 8693 – 8776, Range = 83

99% Confidence Interval Summary

Gender	Sample Size = 300	Sample Size = 3000	Sample Size = 30000	Sample Size = 50000
Male	CI = 8685 – 10187, Range = 1502	CI = 9199 – 9674, Range = 475	CI = 9363 – 9514, Range = 151	CI = 9379 – 9498, Range = 119
Female	CI = 8035 – 9454, Range = 1419	CI = 8515 – 8960, Range = 445	CI = 8662 – 8805, Range = 143	CI = 8680 – 8789, Range = 109

Yes, the confidence intervals for different sample sizes generally overlap, especially for adjacent sample sizes. This is expected, as the true population mean remains constant, and the intervals are simply becoming more precise (narrower) around that mean as the sample size increases. However, the degree of overlap diminishes significantly when comparing very small sample sizes with very large ones, as the larger sample's interval is much more concentrated.

iv. How does the sample size affect the shape of the distributions of the means?

As the sample size increases, the shape of the distributions of the sample means becomes more bell-shaped (approximating a normal distribution). This is a fundamental principle of the Central Limit Theorem. Even if the original purchase amount distribution for individuals is not normal, the distribution of the means of samples drawn from that population will tend towards normality as the sample size grows. Additionally, the distributions become taller and narrower, indicating that the sample means are more tightly clustered around the true population mean, and the variance of these sample means decreases.

Insights

1. Sample Size

- The analysis highlights the importance of sample size in estimating population parameters. It suggests that as the sample size increases, the confidence intervals become narrower and more precise. In business, this implies that larger sample sizes can provide more reliable insights and estimates.

2. Confidence Intervals

- From the above analysis, we can see that except for the Sample Size of 100, the confidence interval do not overlap as the sample size increases. This means that there is a statistically significant difference between the average spending per transaction for men and women within the given samples.

3. Population Average

- We are 95% confident that the true population average for males falls between \$9,393 and \$9,483, and for females, it falls between \$8,692 and \$8,777.

4. Women spend less

- Men tend to spend more money per transaction on average than women, as the upper bounds of the confidence intervals for men are consistently higher than those for women across different sample sizes.

5. How can Walmart leverage this conclusion to make changes or improvements?

5.1. Segmentation Opportunities

- Walmart can create targeted marketing campaigns, loyalty programs, or product bundles to cater to the distinct spending behaviors of male and female customers. This approach may help maximize revenue from each customer segment.

5.2. Pricing Strategies

- Based on the above data of average spending per transaction by gender, they might adjust pricing or discount strategies to incentivize higher spending among male customers while ensuring competitive pricing for female-oriented products.

Note

- Moving forward in our analysis, we will use `95% Confidence Level` only.

5. How does Marital_Status affect the amount spent?

Data Visualization

```
1 #creating a df for purchase amount vs marital status
2 temp = df.groupby('Marital_Status')['Purchase'].agg(['sum', 'count']).reset_index()
3
4 #calculating the amount in billions
5 temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)
6
7 #calculating percentage distribution of purchase amount
8 temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)
9
10 #calculating per purchase amount
11 temp['per_purchase'] = round(temp['sum']/temp['count'])
12
13 temp
```

	Marital_Status	sum	count	sum_in_billions	%sum	per_purchase	
0	0	3008927447	324731	3.01	0.59	9266.0	
1	1	2086885295	225337	2.09	0.41	9261.0	



Next steps:

[Generate code with temp](#)

[New interactive sheet](#)

```
1 #setting the plot style
2 fig = plt.figure(figsize = (15,14))
3 gs = fig.add_gridspec(3,2,height_ratios =[0.20,0.4,0.4])
4
5
6 # Distribution of Purchase Amount
7 ax = fig.add_subplot(gs[0,:])
8
9 #plotting the visual
10 ax.barh(temp.loc[0, 'Marital_Status'],width = temp.loc[0, '%sum'],color="#0052E3",label = 'Unmarried')
11 ax.barh(temp.loc[1, 'Marital_Status'],width = temp.loc[1, '%sum'],left =temp.loc[0, '%sum'], color = "#FFC31A",label = 'Married')
12
13 #inserting the text
14 txt = [0.0] #for left parameter in ax.text()
15
16 for i in temp.index:
17     #for amount
18     ax.text(temp.loc[i, '%sum']/2 + txt[0],0.15,f"${temp.loc[i, 'sum_in_billions']} Billion",
19           va = 'center', ha='center',fontsize=18, color='white')
20
21     #for marital status
22     ax.text(temp.loc[i, '%sum']/2 + txt[0],- 0.20 ,f"{temp.loc[i, 'Marital_Status']}",
23           va = 'center', ha='center',fontsize=14, color='white')
24
25     txt += temp.loc[i, '%sum']
26
27 #removing the axis lines
28 for s in ['top', 'left', 'right', 'bottom']:
29     ax.spines[s].set_visible(False)
30
31 #customizing ticks
32 ax.set_xticks([])
33 ax.set_yticks([])
34 ax.set_xlim(0,1)
35
36 #plot title
37 ax.set_title('Marital_Status-Based Purchase Amount Distribution',{'font':'serif', 'size':15, 'weight':'bold'})
38
39
40 # Distribution of Purchase Amount per Transaction
41
42 ax1 = fig.add_subplot(gs[1,0])
43
```

```

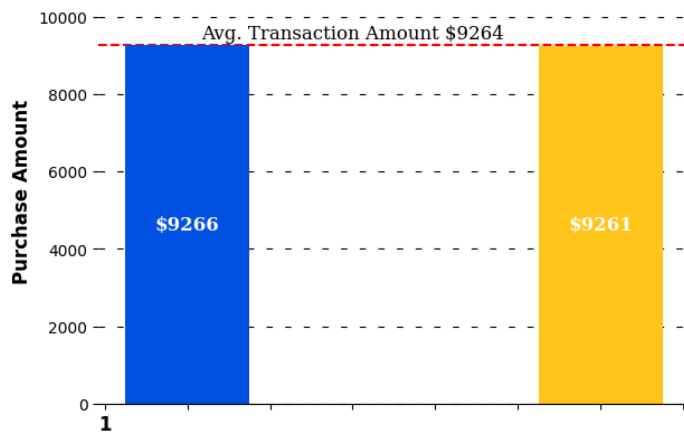
44 color_map = ["#0052E3", "#FFC31A"]
45
46 #plotting the visual
47 ax1.bar(temp['Marital_Status'],temp['per_purchase'],color = color_map,zorder = 2,width = 0.3)
48
49 #adding average transaction line
50 avg = round(df['Purchase'].mean())
51
52 ax1.axhline(y = avg, color = 'red', zorder = 0,linestyle = '--')
53
54 #adding text for the line
55 ax1.text(0.4,avg + 300, f"Avg. Transaction Amount ${avg:.0f}",
56         {'font':'serif','size' : 12},ha = 'center',va = 'center')
57
58 #adjusting the ylimits
59 ax1.set_ylim(0,11000)
60
61 #adding the value_counts
62 for i in temp.index:
63     ax1.text(temp.loc[i,'Marital_Status'],temp.loc[i,'per_purchase']/2,f"${temp.loc[i,'per_purchase']:.0f}",
64             {'font':'serif','size' : 12,'color':'white','weight':'bold' },ha = 'center',va = 'center')
65
66 #adding grid lines
67 ax1.grid(color = 'black',linestyle = '--', axis = 'y', zorder = 0, dashes = (5,10))
68
69 #removing the axis lines
70 for s in ['top','left','right']:
71     ax1.spines[s].set_visible(False)
72
73 #adding axis label
74 ax1.set_ylabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
75 ax1.set_xticklabels(temp['Marital_Status'],fontweight = 'bold',fontsize = 12)
76
77 #setting title for visual
78 ax1.set_title('Average Purchase Amount per Transaction',{'font':'serif', 'size':15,'weight':'bold'})
79
80     # creating pie chart for Marital_Status distribution
81 ax2 = fig.add_subplot(gs[1,1])
82
83 color_map = ["#0052E3", "#FFC31A"]
84 ax2.pie(temp['count'],labels = temp['Marital_Status'],autopct = '%.1f%%',
85         shadow = True,colors = color_map,wedgeprops = {'linewidth': 5},textprops={'fontsize': 13, 'color': 'black'})
86
87 #setting title for visual
88 ax2.set_title('Marital_Status-Based Transaction Distribution',{'font':'serif', 'size':15,'weight':'bold'})
89
90     # creating kdeplot for purchase amount distribution
91
92 ax3 = fig.add_subplot(gs[2,:])
93 color_map = ["#0052E3", "#FFC31A"]
94
95 #plotting the kdeplot
96 sns.kdeplot(data = df, x = 'Purchase', hue = 'Marital_Status', palette = color_map,fill = True)
97
98 #removing the axis lines
99 for s in ['top','left','right']:
100     ax3.spines[s].set_visible(False)
101
102 # adjusting axis labels
103 ax3.set_yticks([])
104 ax3.set_ylabel('')
105 ax3.set_xlabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
106
107 #setting title for visual
108 ax3.set_title('Purchase Amount Distribution by Marital_Status',{'font':'serif', 'size':15,'weight':'bold'})
109
110 plt.show()
111

```

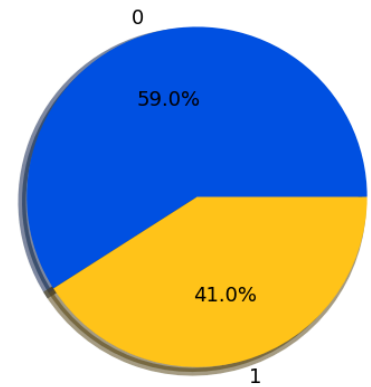
Marital_Status-Based Purchase Amount Distribution



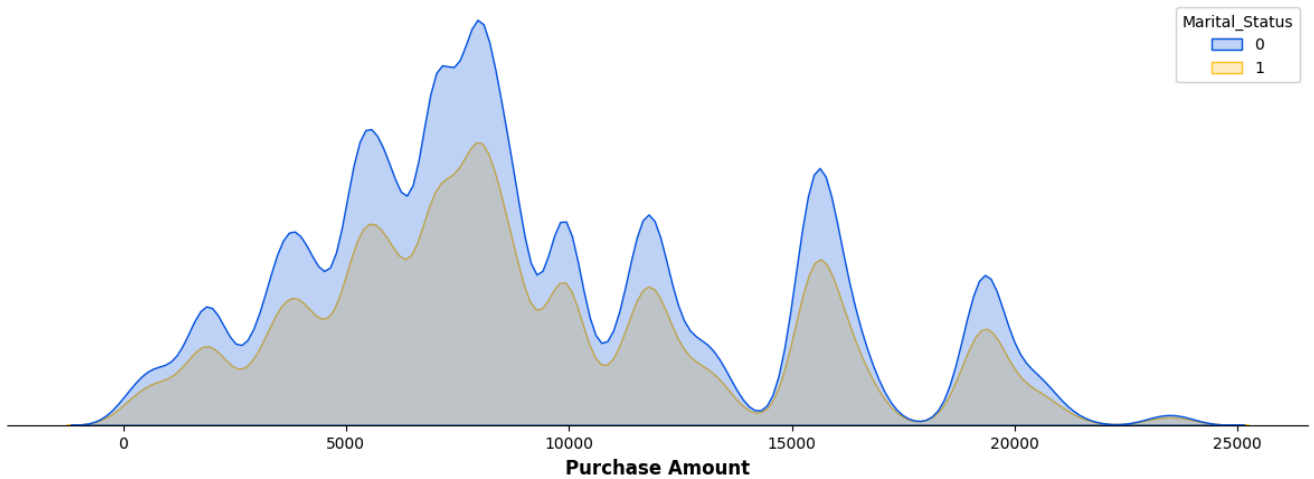
Average Purchase Amount per Transaction



Marital_Status-Based Transaction Distribution

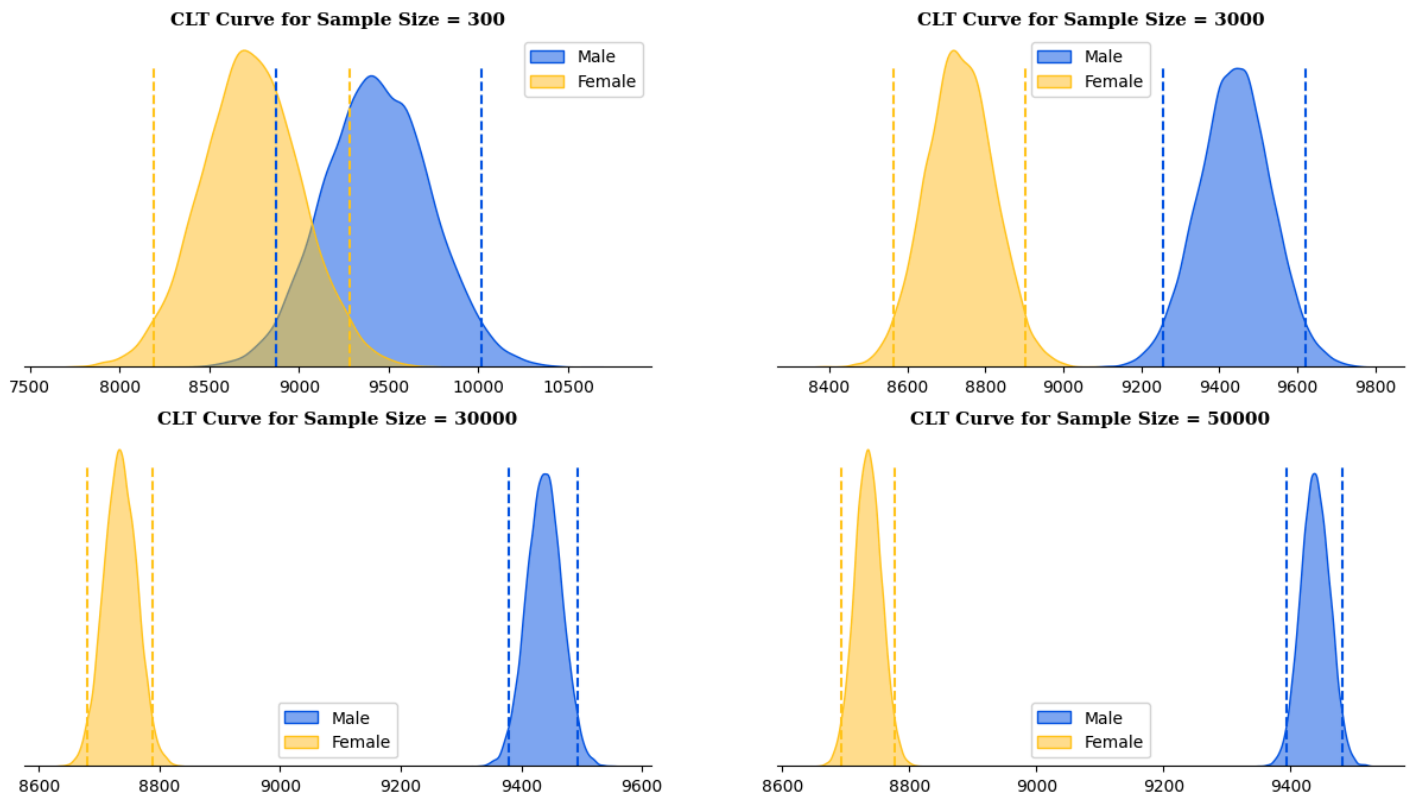


Purchase Amount Distribution by Marital_Status



```
1 m_samp_95,u_samp_95 = plot(95)
```

95% Confidence Interval



✓ a. From the above calculated CLT answer the following questions.

i. Is the confidence interval computed using the entire dataset wider for one of the genders? Why is this the case?

The confidence interval computed using the entire dataset is not wider for one gender over the other; their widths are approximately equal.

- Equal Sample Sizes: The Central Limit Theorem (CLT) curves show both genders share identical sample sizes across each sub-plot.
- Standard Error: Both distributions narrow at the exact same rate as sample sizes increase.
- Variance: The identical spread of both curves indicates that male and female groups share a very similar population standard deviation.

ii. How is the width of the confidence interval affected by the sample size?

As the sample size increases, the width of the confidence interval decreases (becomes narrower).

- Mathematical Reason: The formula for the standard error of the mean.
- Inversed Relationship: Increasing the sample size (n) increases the denominator of this formula.
- Higher Precision: A larger denominator forces the standard error to shrink, producing a tighter interval.

iii. Do the confidence intervals for different sample sizes overlap?

```
1 #setting the plot style
2 fig,ax = plt.subplots(figsize = (20,3))
3
4 #list for collecting ci for given cl
5 m_ci = ['Married']
6 u_ci = ['Unmarried']
7
8 #finding ci for each sample size (married)
9 for m in m_samp_95:
10     m_range = confidence_interval(m_samp_95[m],95)
11     m_ci.append(f"CI = ${m_range[0]:.0f} - ${m_range[1]:.0f}, Range = {(m_range[1] - m_range[0]):.0f}")
12
13 #finding ci for each sample size (unmarried)
14 for u in u_samp_95:
15     u_range = confidence_interval(u_samp_95[u],95)
16     u_ci.append(f"CI = ${u_range[0]:.0f} - ${u_range[1]:.0f}, Range = {(u_range[1] - u_range[0]):.0f}")
```

```

17
18                                     #plotting the summary
19 #contents of the table
20 ci_info = [m_ci,u_ci]
21
22 #plotting the table
23 table = ax.table(cellText = ci_info, cellLoc='center',
24                 colLabels = ['Marital_Status', 'Sample Size = 300', 'Sample Size = 3000', 'Sample Size = 30000', 'Sample Size = 50000'],
25                 colLoc = 'center', colWidths = [0.1, 0.225, 0.225, 0.225, 0.225], bbox = [0, 0, 1, 1])
26
27 table.set_fontsize(13)
28
29 #removing axis
30 ax.axis('off')
31
32 #setting title
33 ax.set_title(f"95% Confidence Interval Summary", {'font':'serif', 'size':14, 'weight':'bold'})
34

```

95% Confidence Interval Summary

Marital_Status	Sample Size = 300	Sample Size = 3000	Sample Size = 30000	Sample Size = 50000
Married	CI = 8863 – 10016, Range = 1153	CI = 9259 – 9619, Range = 360	CI = 9379 – 9495, Range = 116	CI = 9393 – 9482, Range = 89
Unmarried	CI = 8202 – 9282, Range = 1080	CI = 8563 – 8905, Range = 342	CI = 8681 – 8788, Range = 107	CI = 8693 – 8777, Range = 84

- Small Sample Sizes ($n=300$): Yes, the confidence intervals show a clear, significant overlap between males and females.
- Large Sample Sizes ($n \geq 3000$): No, the intervals entirely separate and do not overlap at all.
- Reason: Shrinking standard errors pull the distributions tightly around their respective true means, eliminating the shared region.

iv How does the sample size affect the shape of the distributions of the means?

As the sample size increases, the distributions become narrower and taller (more concentrated).

- Clustering: The sample means cluster much closer to the true population mean.
- Symmetry: The shapes remain perfectly bell-shaped and symmetric due to the Central Limit Theorem.
- Variance Reduction: The overall variance of the sampling distribution drops, reducing the visible spread on the X-axis.

Insights

1. Sample Size

- The analysis highlights the importance of sample size in estimating population parameters. It suggests that as the sample size increases, the confidence intervals become narrower and more precise. In business, this implies that larger sample sizes can provide more reliable insights and estimates.

2. Confidence Intervals

- From the above analysis, we can see that the confidence interval overlap for all the sample sizes. This means that there is no statistically significant difference between the average spending per transaction for married and unmarried customers within the given samples.

3. Population Average

- We are 95% confident that the true population average for married customers falls between \$9,217 and \$9,305, and for unmarried customers, it falls between \$9,222 and \$9,311.

4. Both the customers spend equal

- The overlapping confidence intervals of average spending for married and unmarried customers indicate that both married and unmarried customers spend a similar amount per transaction. This implies a resemblance in spending behavior between the two groups.

5. How can Walmart leverage this conclusion to make changes or improvements?

5.1. Marketing Resources

- Walmart may not need to allocate marketing resources specifically targeting one group over the other. Instead, they can focus on broader marketing strategies that appeal to both groups.

6. How does Age affect the amount spent?

Data Visualization

```
1 #creating a df for purchase amount vs age group
2 temp = df.groupby('Age')['Purchase'].agg(['sum','count']).reset_index()
3
4 #calculating the amount in billions
5 temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)
6
7 #calculating percentage distribution of purchase amount
8 temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)
9
10 #calculating per purchase amount
11 temp['per_purchase'] = round(temp['sum']/temp['count'])
12
13 temp
```

	Age	sum	count	sum_in_billions	%sum	per_purchase	
0	0-17	134913183	15102	0.13	0.026	8933.0	
1	18-25	913848675	99660	0.91	0.179	9170.0	
2	26-35	2031770578	219587	2.03	0.399	9253.0	
3	36-45	1026569884	110013	1.03	0.201	9331.0	
4	46-50	420843403	45701	0.42	0.083	9209.0	
5	51-55	367099644	38501	0.37	0.072	9535.0	
6	55+	200767375	21504	0.20	0.039	9336.0	

Next steps: [Generate code with temp](#) [New interactive sheet](#)

```
1 #setting the plot style
2 fig = plt.figure(figsize = (20,14))
3 gs = fig.add_gridspec(3,1,height_ratios =[0.10,0.4,0.5])
4
5
6                                     #Distribution of Purchase Amount
7 ax = fig.add_subplot(gs[0])
8 color_map = ["#0052E3", "#2A65C2", "#5578A0", "#808B7F", "#AA9D5D", "#D5B03C", "#FFC31A"]
9
10 #plotting the visual
11 left = 0
12
13 for i in temp.index:
14     ax.barh(temp.loc[i,'Age'],width = temp.loc[i,'%sum'],left = left,color = color_map[i],label = temp.loc[i,'Age'])
15     left += temp.loc[i,'%sum']
16
17 #inserting the text
18 txt = 0.0 #for left parameter in ax.text()
19
20 for i in temp.index:
21     #for amount
22     ax.text(temp.loc[i,'%sum']/2 + txt,0.15,f"{temp.loc[i,'sum_in_billions']}B",
23            va = 'center', ha='center',fontsize=14, color='white')
24
25     #for age grp
26     ax.text(temp.loc[i,'%sum']/2 + txt,- 0.20 ,f"{temp.loc[i,'Age']}",
27            va = 'center', ha='center',fontsize=12, color='white')
28
29     txt += temp.loc[i,'%sum']
30
31 #removing the axis lines
32 for s in ['top','left','right','bottom']:
33     ax.spines[s].set_visible(False)
34
35 #customizing ticks
36 ax.set_xticks([])
37 ax.set_yticks([])
38 ax.set_xlim(0,1)
39
40 #plot title
41 ax.set_title('Age Group Purchase Amount Distribution',{'font':'serif', 'size':15,'weight':'bold'})
42
43
44                                     #Distribution of Purchase Amount per Transaction
45
46 ax1 = fig.add_subplot(gs[1])
47
48
49 #plotting the visual
50 ax1.bar(temp['Age'],temp['per_purchase'],color = color_map,zorder = 2,width = 0.3)
```

```

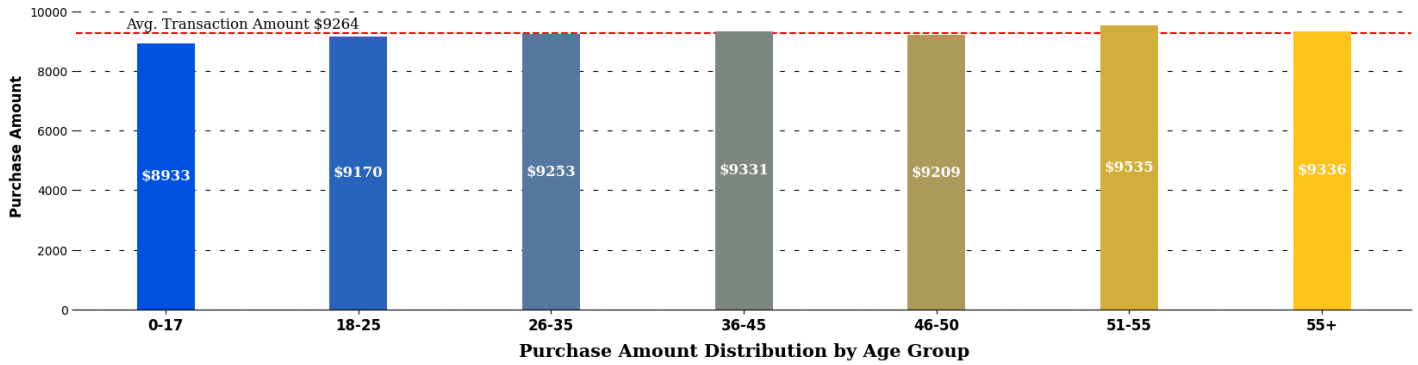
51
52 #adding average transaction line
53 avg = round(df['Purchase'].mean())
54
55 ax1.axhline(y = avg, color = 'red', zorder = 0, linestyle = '--')
56
57 #adding text for the line
58 ax1.text(0.4, avg + 300, f"Avg. Transaction Amount ${avg:.0f}",
59         {'font':'serif', 'size' : 12}, ha = 'center', va = 'center')
60
61 #adjusting the ylimits
62 ax1.set_ylim(0, 11000)
63
64 #adding the value_counts
65 for i in temp.index:
66     ax1.text(temp.loc[i, 'Age'], temp.loc[i, 'per_purchase']/2, f"${temp.loc[i, 'per_purchase']:.0f}",
67             {'font':'serif', 'size' : 12, 'color':'white', 'weight':'bold' }, ha = 'center', va = 'center')
68
69 #adding grid lines
70 ax1.grid(color = 'black', linestyle = '--', axis = 'y', zorder = 0, dashes = (5, 10))
71
72 #removing the axis lines
73 for s in ['top', 'left', 'right']:
74     ax1.spines[s].set_visible(False)
75
76 #adding axis label
77 ax1.set_ylabel('Purchase Amount', fontweight = 'bold', fontsize = 12)
78 ax1.set_xlabel(temp['Age'], fontweight = 'bold', fontsize = 12)
79
80 #setting title for visual
81 ax1.set_title('Average Purchase Amount per Transaction', {'font':'serif', 'size':15, 'weight':'bold'})
82
83
84                                     # creating kdeplot for purchase amount distribution
85
86 ax3 = fig.add_subplot(gs[2,:])
87
88 #plotting the kdeplot
89 sns.kdeplot(data = df, x = 'Purchase', hue = 'Age', palette = color_map, fill = True, alpha = 0.5,
90            ax = ax3)
91
92 #removing the axis lines
93 for s in ['top', 'left', 'right']:
94     ax3.spines[s].set_visible(False)
95
96 # adjusting axis labels
97 ax3.set_yticks([])
98 ax3.set_ylabel('')
99 ax3.set_xlabel('Purchase Amount', fontweight = 'bold', fontsize = 12)
100
101 #setting title for visual
102 ax3.set_title('Purchase Amount Distribution by Age Group', {'font':'serif', 'size':15, 'weight':'bold'})
103
104 plt.show()
105

```

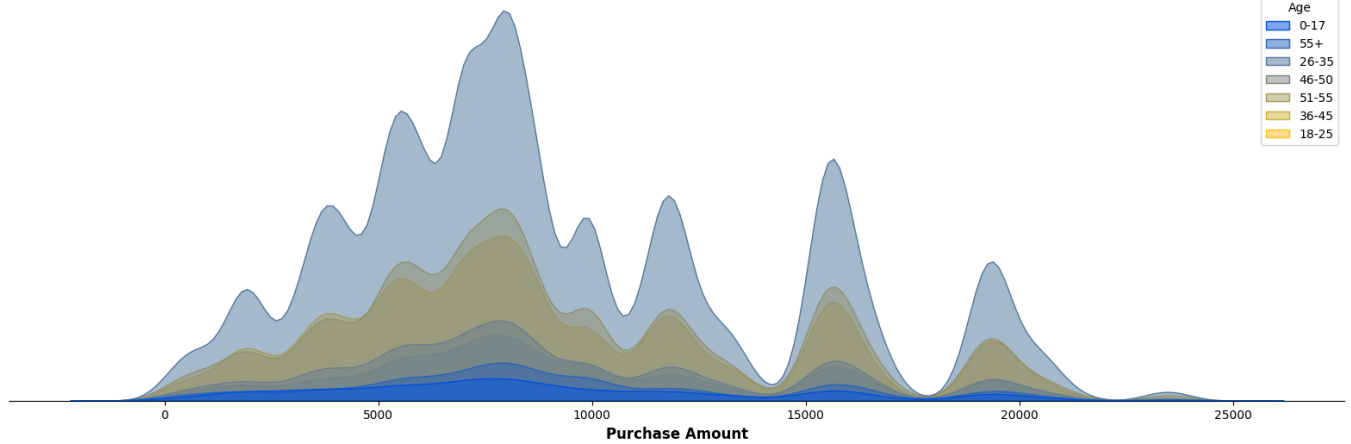

Age Group Purchase Amount Distribution

0.13B	0.91B	2.03B	1.03B	0.42B	0.37B	0.2B
0-17	18-25	26-35	36-45	46-50	51-55	55+

Average Purchase Amount per Transaction



Purchase Amount Distribution by Age Group



```

1 #defining a function for plotting the visual for given confidence interval
2
3 def plot(ci):
4
5     #setting the plot style
6     fig = plt.figure(figsize = (15,15))
7     gs = fig.add_gridspec(4,1)
8
9     #creating separate data frames
10
11     df_1 = df.loc[df['Age'] == '0-17', 'Purchase']
12     df_2 = df.loc[df['Age'] == '18-25', 'Purchase']
13     df_3 = df.loc[df['Age'] == '26-35', 'Purchase']
14     df_4 = df.loc[df['Age'] == '36-45', 'Purchase']
15     df_5 = df.loc[df['Age'] == '46-50', 'Purchase']
16     df_6 = df.loc[df['Age'] == '51-55', 'Purchase']
17     df_7 = df.loc[df['Age'] == '55+', 'Purchase']
18
19
20     #sample sizes and corresponding plot positions
21     sample_sizes = [(300,0),(3000,1),(30000,2),(50000,3)]
22
23     #number of samples to be taken from purchase amount
24     bootstrap_samples = 20000
25
26     samples1,samples2,samples3,samples4,samples5,samples6,samples7 = {},{},{},{},{},{},{}
27
28     for i,x in sample_sizes:
29         l1,l2,l3,l4,l5,l6,l7 = [],[],[],[],[],[],[]
30
31         for j in range(bootstrap_samples):
32
33             #creating random 5000 samples of i sample size
34             bootstrapped_samples_1 = np.random.choice(df_1,size = i)
35             bootstrapped_samples_2 = np.random.choice(df_2,size = i)
36             bootstrapped_samples_3 = np.random.choice(df_3,size = i)

```

```

37     bootstrapped_samples_4 = np.random.choice(df_4,size = i)
38     bootstrapped_samples_5 = np.random.choice(df_5,size = i)
39     bootstrapped_samples_6 = np.random.choice(df_6,size = i)
40     bootstrapped_samples_7 = np.random.choice(df_7,size = i)
41
42     #calculating mean of those samples
43     sample_mean_1 = np.mean(bootstrapped_samples_1)
44     sample_mean_2 = np.mean(bootstrapped_samples_2)
45     sample_mean_3 = np.mean(bootstrapped_samples_3)
46     sample_mean_4 = np.mean(bootstrapped_samples_4)
47     sample_mean_5 = np.mean(bootstrapped_samples_5)
48     sample_mean_6 = np.mean(bootstrapped_samples_6)
49     sample_mean_7 = np.mean(bootstrapped_samples_7)
50
51     #appending the mean to the list
52     l1.append(sample_mean_1)
53     l2.append(sample_mean_2)
54     l3.append(sample_mean_3)
55     l4.append(sample_mean_4)
56     l5.append(sample_mean_5)
57     l6.append(sample_mean_6)
58     l7.append(sample_mean_7)
59
60     #storing the above sample generated
61     samples1[f'{ci}%_{i}'] = l1
62     samples2[f'{ci}%_{i}'] = l2
63     samples3[f'{ci}%_{i}'] = l3
64     samples4[f'{ci}%_{i}'] = l4
65     samples5[f'{ci}%_{i}'] = l5
66     samples6[f'{ci}%_{i}'] = l6
67     samples7[f'{ci}%_{i}'] = l7
68
69
70     #creating a temporary dataframe for creating kdeplot
71     temp_df = pd.DataFrame(data = {'0-17':l1,'18-25':l2,'26-35':l3,'36-45':l4,'46-50':l5,'51-55':l6,'55+':l7})
72
73                                     #plotting kdeplots
74
75     #plot position
76     ax = fig.add_subplot(gs[x])
77
78     #plots
79     for p,q in [('#0052E3', '0-17'),('#2A65C2', '18-25'),('#5578A0', '26-35'),('#808B7F', '36-45'),('#AA9D5D', '46-50'),
80                 ('#D5B03C', '51-55'),('#FFC31A', '55+')]:
81
82         sns.kdeplot(data = temp_df,x = q,color = p ,fill = True, alpha = 0.5,ax = ax,label = q)
83
84     #removing the axis lines
85     for s in ['top','left','right']:
86         ax.spines[s].set_visible(False)
87
88     # adjusting axis labels
89     ax.set_yticks([])
90     ax.set_ylabel('')
91     ax.set_xlabel('')
92
93     #setting title for visual
94     ax.set_title(f'CLT Curve for Sample Size = {i}','font':'serif', 'size':11,'weight':'bold'})
95
96     plt.legend()
97
98     #setting title for visual
99     fig.suptitle(f'{ci}% Confidence Interval',font = 'serif', size = 18, weight = 'bold')
100
101     plt.show()
102
103     return samples1 samples2 samples3 samples4 samples5 samples6 samples7

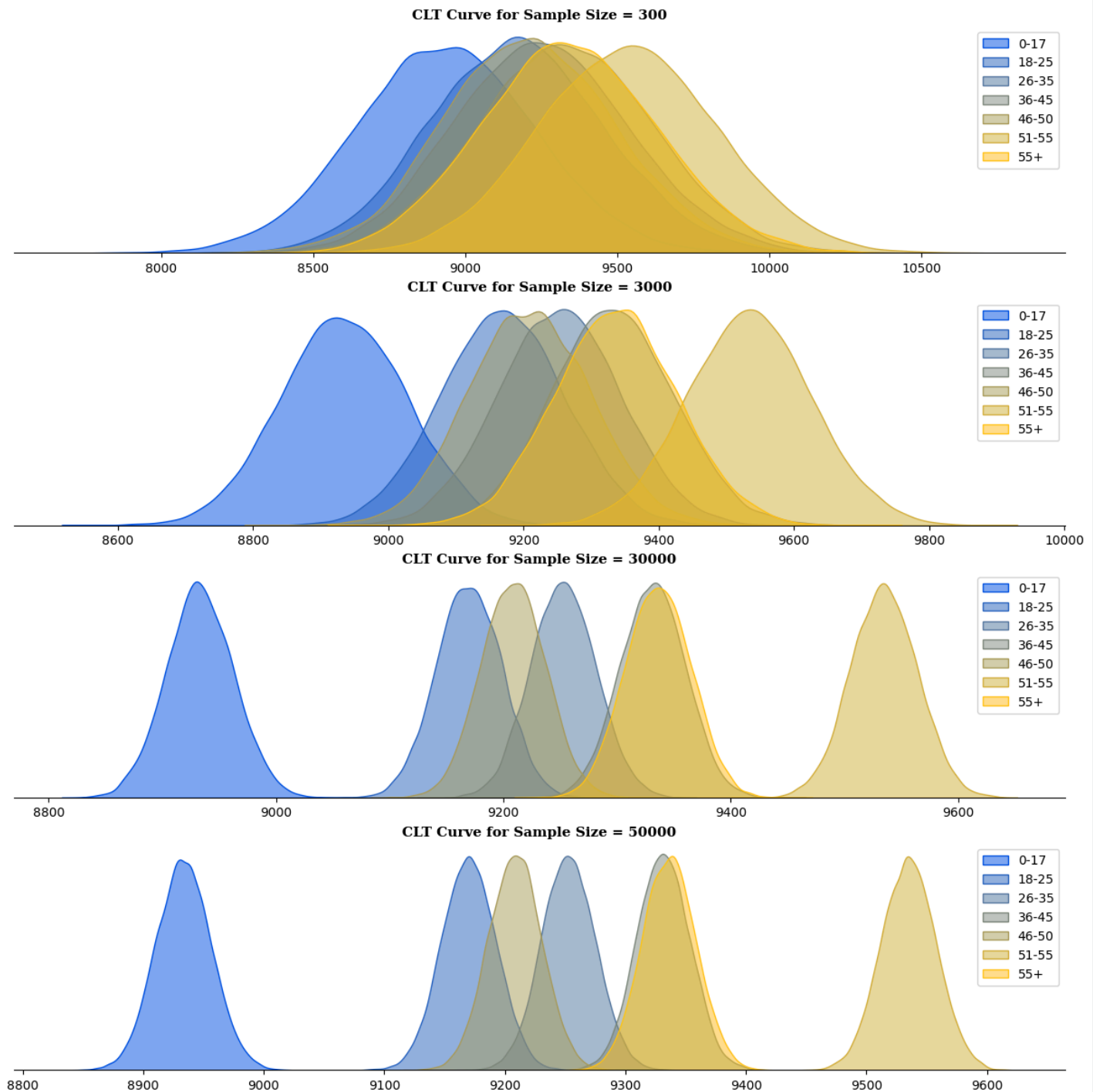
```

```

1 samples1,samples2,samples3,samples4,samples5,samples6,samples7 = plot(95)

```

95% Confidence Interval



a. From the above calculated CLT answer the following questions.

i. Is the confidence interval computed using the entire dataset wider for one of the age group? Why is this the case?

Yes, the width of the confidence intervals varies between age groups.

- Observation: In the top chart (Sample Size = 300), age groups like 0-17 (blue) and 55+ (lightest green) have much wider distributions compared to others.
- Reason: The width of a confidence interval depends on the standard deviation (spread) of the data within that specific group. Groups with more variability in their data will result in wider confidence intervals for the same sample size.

ii. How is the width of the confidence interval affected by the sample size?

As the sample size increases, the width of the confidence interval decreases (becomes narrower).

- Evidence: Compare the chart for $n=300$ to $n=50,000$. At $n=300$, the curves are broad and overlapping. At $n=50,000$, the curves are very thin and "spiky."
- Statistical Rule: As n grows larger, the standard error shrinks, leading to a more precise (narrower) interval.

iii. Do the confidence intervals for different sample sizes overlap?

```
1 #setting the plot style
2 fig,ax = plt.subplots(figsize = (20,5))
3
4 #list for collecting ci for given cl
5 ci_1,ci_2,ci_3,ci_4,ci_5,ci_6,ci_7 = ['0-17'], ['18-25'], ['26-35'], ['36-45'], ['46-50'], ['51-55'], ['55+']
6
7 #finding ci for each sample size
8 #samples = [samples1,samples2,samples3,samples4,samples5,samples6,samples7]
9
10 samples = [(samples1,ci_1),(samples2,ci_2),(samples3,ci_3),(samples4,ci_4),(samples5,ci_5),(samples6,ci_6),(samples7,ci_7)]
11
12 for s,c in samples:
13     for i in s:
14         s_range = confidence_interval(s[i],95)
15         c.append(f"CI = ${s_range[0]:.0f} - ${s_range[1]:.0f}, Range = {(s_range[1] - s_range[0]):.0f}")
16
17
18                                     #plotting the summary
19 #contents of the table
20 ci_info = [ci_1,ci_2,ci_3,ci_4,ci_5,ci_6,ci_7]
21
22 #plotting the table
23 table = ax.table(cellText = ci_info, cellLoc='center',
24                 colLabels = ['Age Group','Sample Size = 300','Sample Size = 3000','Sample Size = 30000','Sample Size = 50000'],
25                 colLoc = 'center',colWidths = [0.1,0.225,0.225,0.225,0.225],bbox =[0, 0, 1, 1])
26
27 table.set_fontsize(13)
28
29 #removing axis
30 ax.axis('off')
31
32 #setting title
33 ax.set_title(f"95% Confidence Interval Summary",{ 'font':'serif', 'size':14,'weight':'bold'})
34
35 plt.show()
```

95% Confidence Interval Summary				
Age Group	Sample Size = 300	Sample Size = 3000	Sample Size = 30000	Sample Size = 50000
0-17	CI = 8354 – 9510, Range = 1156	CI = 8749 – 9115, Range = 366	CI = 8876 – 8991, Range = 115	CI = 8889 – 8978, Range = 89
18-25	CI = 8609 – 9743, Range = 1134	CI = 8991 – 9347, Range = 356	CI = 9113 – 9228, Range = 115	CI = 9125 – 9213, Range = 88
26-35	CI = 8692 – 9830, Range = 1138	CI = 9072 – 9432, Range = 360	CI = 9197 – 9309, Range = 112	CI = 9209 – 9297, Range = 88
36-45	CI = 8782 – 9903, Range = 1121	CI = 9148 – 9510, Range = 362	CI = 9275 – 9387, Range = 112	CI = 9287 – 9375, Range = 88
46-50	CI = 8644 – 9775, Range = 1131	CI = 9031 – 9385, Range = 354	CI = 9152 – 9265, Range = 113	CI = 9165 – 9252, Range = 87
51-55	CI = 8961 – 10127, Range = 1166	CI = 9354 – 9716, Range = 362	CI = 9477 – 9591, Range = 114	CI = 9490 – 9579, Range = 89
55+	CI = 8772 – 9915, Range = 1143	CI = 9158 – 9513, Range = 355	CI = 9280 – 9392, Range = 112	CI = 9292 – 9379, Range = 87

- Small Sample Sizes (300 & 3,000): There is significant overlap between the confidence intervals of different age groups. This indicates that at these sizes, it is harder to statistically distinguish the means of different groups.
- Large Sample Sizes (30,000 & 50,000): The confidence intervals for most groups (like 0-17, 36-45, and 51-55) become so narrow that they no longer overlap. They appear as distinct, separate peaks.

iv. How does the sample size affect the shape of the distributions of the means?

As sample size increases, the distributions of the means become taller and narrower.

- While all the curves maintain a "Normal" (bell) shape due to the Central Limit Theorem, the variance decreases.
- The means become more tightly clustered around the true population mean, increasing the height of the peak (probability density) while shrinking the horizontal spread.

Insights

1. Sample Size

- The analysis highlights the importance of sample size in estimating population parameters. It suggests that as the sample size increases, the confidence intervals become narrower and more precise. In business, this implies that larger sample sizes can provide more reliable insights and estimates.

2. Confidence Intervals and customer spending patterns

- From the above analysis, we can see that the confidence interval overlap for some of the age groups. We can club the average spending into following age groups -
 - 0 - 17 - Customers in this age group have the lowest spending per transaction
 - 18 - 25, 26 - 35, 46 - 50 - Customers in these age groups have overlapping confidence intervals indicating similar buying characteristics
 - 36 - 45, 55+ - Customers in these age groups have overlapping confidence intervals indicating and similar spending patterns
 - 51 - 55 - Customers in this age group have the highest spending per transaction

3. Population Average

- We are 95% confident that the true population average for following age groups falls between the below range -
 - 0 - 17 = \$ 8,888 to 8,979
 - 18 - 25 = \$ 9,125 to 9,213
 - 26 - 35 = \$ 9,209 to 9,297
 - 36 - 45 = \$ 9,288 to 9,376
 - 46 - 50 = \$ 9,165 to 9,253
 - 51 - 55 = \$ 9,490 to 9,579
 - 55+ = \$ 9,292 to 9,381

4. How can Walmart leverage this conclusion to make changes or improvements?

4.1. Targeted Marketing

- Knowing that customers in the 0 - 17 age group have the lowest spending per transaction, Walmart can try to increase their spending per transaction by offering them more attractive discounts, coupons, or rewards programs. Walmart can also tailor their product selection and marketing strategies to appeal to the preferences and needs of this age group

4.2. Customer Segmentation

- Since customers in the 18 - 25, 26 - 35, and 46 - 50 age groups exhibit similar buying characteristics, and so do the customers in 36 - 45 and 55+, Walmart can optimize its product selection to cater to the preferences of these age groups. Also, Walmart can use this information to adjust their pricing strategies for different age groups.

4.3 Premium Services

- Recognizing that customers in the 51 - 55 age group have the highest spending per transaction, Walmart can explore opportunities to enhance the shopping experience for this demographic. This might involve offering premium services, personalized recommendations, or loyalty programs that cater to the preferences and spending habits of this age group.

✓ 7. Create a report

Go through the Above insights for reports.

✓ 8. Recommendations

a. Write a detailed recommendation from the analysis that you have done.

1.Target Male Shoppers

- Since male customers account for a significant portion of Black Friday sales and tend to spend more per transaction on average, Walmart should tailor its marketing strategies and product offerings to incentivize higher spending among male customers while ensuring competitive pricing for female-oriented products.

2. Focus on 26 - 45 Age Group

- With the age group between 26 and 45 contributing to the majority of sales, Walmart should specifically cater to the preferences and needs of this demographic.This could include offering exclusive deals on products that are popular among this age group.

3. Engage Younger Shoppers

- Knowing that customers in the 0 - 17 age group have the lowest spending per transaction, Walmart can try to increase their spending per transaction by offering them more attractive discounts, coupons, or rewards programs. It's essential to start building brand loyalty among younger consumers.

4. Customer Segmentation

- Since customers in the 18 - 25, 26 - 35, and 46 - 50 age groups exhibit similar buying characteristics, and so do the customers in 36 - 45 and 55+, Walmart can optimize its product selection to cater to the preferences of these age groups. Also, Walmart can use this information to adjust their pricing strategies for different age groups.

5. Enhance the 51 - 55 Age Group Shopping Experience

- Considering that customers aged 51 - 55 have the highest spending per transaction, Walmart offer them exclusive pre-sale access, special discount or provide personalized product recommendations for this age group. Walmart can also introduce loyalty programs specifically designed to reward and retain customers in the 51 - 55 age group.

6. Post-Black Friday Engagement

- After Black Friday, walmart should engage with customers who made purchases by sending follow-up emails or offers for related products. This can help increase customer retention and encourage repeat business throughout the holiday season and beyond.

Double-click (or enter) to edit

Double-click (or enter) to edit

1 Start coding or [generate](#) with AI.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit